

Tarea Semana 5: Calculadora Científica con Tests

Pruebas Unitarias y Testing

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido a la Semana 5! Has aprendido a programar, tomar decisiones, usar bucles, manejar errores y trabajar con librerías. Ahora es momento de aprender una habilidad profesional crucial: **testing**.

¿Cómo sabes que tu código funciona correctamente? ¿Cómo garantizas que funciona en todos los casos posibles? ¿Cómo te aseguras que un cambio no rompe funcionalidad existente? La respuesta es: **pruebas unitarias**.

Las pruebas unitarias son código que verifica que tu código funciona correctamente. Los programadores profesionales escriben tests para cada función que crean. En esta tarea, aprenderás a usar **pytest**, el framework de testing más popular de Python, para crear pruebas exhaustivas de una calculadora científica.

2. Objetivos de Aprendizaje

Al completar esta tarea, serás capaz de:

- Entender la importancia del testing en desarrollo de software
- Instalar y configurar **pytest**
- Escribir funciones de prueba con **pytest**
- Usar la función **assert** para verificar resultados
- Crear pruebas para casos normales, límite y excepciones
- Organizar tests en archivos separados
- Ejecutar tests y entender reportes de **pytest**
- Aplicar Test-Driven Development (TDD) básico
- Calcular cobertura de código con **pytest-cov**
- Identificar y corregir bugs usando tests

3. Enunciado del Proyecto

3.1. Descripción General

Vas a crear una **Calculadora Científica** completa con operaciones matemáticas avanzadas, Y crear pruebas unitarias exhaustivas para cada función usando **pytest**.

Este proyecto tiene DOS partes iguales en importancia:

1. Implementar las funciones de la calculadora
2. Escribir tests completos para cada función

3.2. Filosofía TDD (Test-Driven Development)

Opcionalmente, puedes aplicar TDD:

1. Escribir el test PRIMERO (que falla porque la función no existe)
2. Implementar la función mínima para que pase
3. Refactorizar si es necesario
4. Repetir

4. Requisitos Funcionales

4.1. Parte 1: Funciones a Implementar

Debes crear un archivo `calculadora.py` con las siguientes funciones:

4.1.1. Operaciones Básicas

Listing 1: Operaciones basicas

```
1 def sumar(a, b):
2     """Suma dos numeros"""
3     return a + b
4
5 def restar(a, b):
6     """Resta b de a"""
7     return a - b
8
9 def multiplicar(a, b):
10    """Multiplica dos numeros"""
11    return a * b
12
13 def dividir(a, b):
14    """Divide a entre b
15    Raises:
16        ValueError: Si b es 0
17    """
18    if b == 0:
19        raise ValueError("No se puede dividir entre cero")
20    return a / b
```

4.1.2. Operaciones Avanzadas

Listing 2: Operaciones avanzadas

```
1 def potencia(base, exponente):
2     """Calcula base elevado a exponente"""
3     return base ** exponente
4
5 def raiz_cuadrada(numero):
6     """Calcula la raiz cuadrada
7     Raises:
8         ValueError: Si numero es negativo
9     """
10    if numero < 0:
11        raise ValueError("No se puede raiz de numero negativo")
12    return numero ** 0.5
13
14 def factorial(n):
15     """Calcula el factorial de n
16     Raises:
17         ValueError: Si n es negativo o no es entero
18     """
19    if not isinstance(n, int):
20        raise ValueError("n debe ser un entero")
21    if n < 0:
22        raise ValueError("n debe ser positivo")
23    if n == 0 or n == 1:
24        return 1
25    resultado = 1
26    for i in range(2, n + 1):
27        resultado *= i
28    return resultado
29
30 def es_primo(n):
31     """Verifica si n es un numero primo
32     Returns:
33         bool: True si es primo, False si no
34     Raises:
35         ValueError: Si n no es entero positivo
36     """
37    if not isinstance(n, int) or n < 2:
38        raise ValueError("n debe ser entero mayor o igual a 2")
39    if n == 2:
40        return True
41    if n % 2 == 0:
42        return False
43    for i in range(3, int(n**0.5) + 1, 2):
44        if n % i == 0:
45            return False
46    return True
```

4.1.3. Operaciones con Listas

Listing 3: Operaciones con listas

```
1 def promedio(numeros):
2     """Calcula el promedio de una lista
3     Raises:
4         ValueError: Si la lista esta vacia
5     """
6     if not numeros:
7         raise ValueError("La lista no puede estar vacia")
8     return sum(numeros) / len(numeros)
9
10 def mediana(numeros):
11     """Calcula la mediana de una lista
12     Raises:
13         ValueError: Si la lista esta vacia
14     """
15     if not numeros:
16         raise ValueError("La lista no puede estar vacia")
17     ordenados = sorted(numeros)
18     n = len(ordenados)
19     if n % 2 == 0:
20         return (ordenados[n//2 - 1] + ordenados[n//2]) / 2
21     return ordenados[n//2]
22
23 def maximo(numeros):
24     """Retorna el maximo de una lista
25     Raises:
26         ValueError: Si la lista esta vacia
27     """
28     if not numeros:
29         raise ValueError("La lista no puede estar vacia")
30     return max(numeros)
31
32 def minimo(numeros):
33     """Retorna el minimo de una lista
34     Raises:
35         ValueError: Si la lista esta vacia
36     """
37     if not numeros:
38         raise ValueError("La lista no puede estar vacia")
39     return min(numeros)
```

4.2. Parte 2: Tests a Implementar

Debes crear un archivo `test_calculadora.py` con tests para TODAS las funciones:

4.2.1. Estructura de un Test

Listing 4: Ejemplo de test basico

```
1 import pytest
2 from calculadora import sumar, dividir, raiz_cuadrada
3
4 # Test simple
```

```
5 def test_sumar_dos_positivos():
6     """Verifica suma de dos numeros positivos"""
7     resultado = sumar(2, 3)
8     assert resultado == 5
9
10 # Test con multiples casos
11 def test_sumar_varios_casos():
12     """Verifica suma en varios escenarios"""
13     assert sumar(0, 0) == 0
14     assert sumar(-1, 1) == 0
15     assert sumar(-5, -3) == -8
16     assert sumar(100, 200) == 300
17
18 # Test de excepciones
19 def test_dividir_entre_cero_lanza_error():
20     """Verifica que dividir entre 0 lanza ValueError"""
21     with pytest.raises(ValueError):
22         dividir(10, 0)
23
24 # Test con mensaje de error especifico
25 def test_raiz_negativo_mensaje_error():
26     """Verifica el mensaje de error especifico"""
27     with pytest.raises(ValueError, match="No se puede raiz"):
28         raiz_cuadrada(-4)
```

4.2.2. Tests Requeridos para cada Función

Para CADA función debes crear al menos:

1. **Test de caso normal:** Entrada típica esperada
2. **Test de caso límite:** Valores en los bordes (0, 1, -1, máximos, mínimos)
3. **Test de excepciones:** Verificar que se lanzan errores apropiados
4. **Test de tipos:** Verificar comportamiento con diferentes tipos

4.2.3. Ejemplo Completo: Tests para factorial

Listing 5: Tests completos para factorial

```
1 import pytest
2 from calculadora import factorial
3
4 # Casos normales
5 def test_factorial_de_0():
6     """Factorial de 0 debe ser 1"""
7     assert factorial(0) == 1
8
9 def test_factorial_de_1():
10     """Factorial de 1 debe ser 1"""
11     assert factorial(1) == 1
12
13 def test_factorial_de_5():
```

```
14     """Factorial de 5 debe ser 120"""
15     assert factorial(5) == 120
16
17 def test_factorial_de_10():
18     """Factorial de 10 debe ser 3628800"""
19     assert factorial(10) == 3628800
20
21 # Casos limite
22 def test_factorial_numero_grande():
23     """Factorial de numero grande"""
24     resultado = factorial(20)
25     assert resultado == 2432902008176640000
26
27 # Casos de error
28 def test_factorial_negativo_lanza_error():
29     """Factorial de negativo debe lanzar ValueError"""
30     with pytest.raises(ValueError, match="debe ser positivo"):
31         factorial(-5)
32
33 def test_factorial_float_lanza_error():
34     """Factorial de float debe lanzar ValueError"""
35     with pytest.raises(ValueError, match="debe ser un entero"):
36         factorial(5.5)
37
38 def test_factorial_string_lanza_error():
39     """Factorial de string debe lanzar ValueError"""
40     with pytest.raises(ValueError):
41         factorial("cinco")
42
43 # Tests parametrizados (avanzado)
44 @pytest.mark.parametrize("n, esperado", [
45     (0, 1),
46     (1, 1),
47     (2, 2),
48     (3, 6),
49     (4, 24),
50     (5, 120),
51 ])
52 def test_factorial_valores_conocidos(n, esperado):
53     """Verifica factorial para valores conocidos"""
54     assert factorial(n) == esperado
```

4.3. Parte 3: Ejecutar Tests

4.3.1. Instalación de pytest

Listing 6: Instalacion de pytest

```
1 # Instalar pytest
2 pip install pytest
3
4 # Instalar pytest-cov para cobertura
5 pip install pytest-cov
```

4.3.2. Ejecutar Tests

Listing 7: Comandos de pytest

```
1 # Ejecutar todos los tests
2 pytest
3
4 # Ejecutar con mas detalle
5 pytest -v
6
7 # Ejecutar tests de un archivo especifico
8 pytest test_calculadora.py
9
10 # Ejecutar un test especifico
11 pytest test_calculadora.py::test_sumar_dos_positivos
12
13 # Ejecutar con cobertura de codigo
14 pytest --cov=calculadora --cov-report=html
15
16 # Ver que tests pasaron/fallaron
17 pytest --tb=short
```

4.3.3. Salida Esperada

Listing 8: Ejemplo de salida de pytest

```
1 ===== test session starts
2   =====
3 platform win32 -- Python 3.11.5, pytest-7.4.0
4 collected 45 items
5
6 test_calculadora.py::test_sumar_dos_positivos PASSED [ 2%]
7 test_calculadora.py::test_sumar_negativos PASSED [ 4%]
8 test_calculadora.py::test_restar_basico PASSED [ 6%]
9 test_calculadora.py::test_multiplicar_por_cero PASSED [ 8%]
10 test_calculadora.py::test_dividir_normales PASSED [11%]
11 test_calculadora.py::test_dividir_entre_cero PASSED [13%]
12 test_calculadora.py::test_potencia_basica PASSED [15%]
13 test_calculadora.py::test_raiz_cuadrada_4 PASSED [17%]
14 test_calculadora.py::test_raiz_negativo_error PASSED [20%]
15 test_calculadora.py::test_factorial_5 PASSED [22%]
```

```

15 test_calculadora.py::test_factorial_negativo_error PASSED [24%]
16 test_calculadora.py::test_es_primo_2 PASSED [26%]
17 test_calculadora.py::test_es_primo_17 PASSED [28%]
18 test_calculadora.py::test_es_primo_4_no_primo PASSED [31%]
19 test_calculadora.py::test_promedio_lista PASSED [33%]
20 test_calculadora.py::test_promedio_vacio_error PASSED [35%]
21 test_calculadora.py::test_mediana_lista_impar PASSED [37%]
22 test_calculadora.py::test_mediana_lista_par PASSED [40%]
23 ... (mas tests) ...
24
25 ===== 45 passed in 0.15s
26 =====
27
28 ----- coverage: platform win32, python 3.11.5 -----
29 Name          Stmts   Miss  Cover
30 ----
31 calculadora.py      85      0  100%
32 ----
33 TOTAL              85      0  100%

```


5. Requisitos Técnicos Detallados

5.1. Estructura del Proyecto

Listing 9: Estructura de archivos

```
1 calculadora_proyecto/  
2   calculadora.py           # Funciones de la calculadora  
3   test_calculadora.py      # Tests con pytest  
4   requirements.txt         # Dependencias  
5   README.txt              # Documentacion  
6   htmlcov/                # Reporte de cobertura (generado)  
7       index.html
```

5.2. Archivo requirements.txt

Listing 10: requirements.txt

```
1 pytest>=7.4.0  
2 pytest-cov>=4.1.0
```

5.3. Cantidad Mínima de Tests

Debes crear AL MENOS:

- **40 tests diferentes** (funciones que empiezan con `test_`)
- **Tests para todas las funciones** implementadas
- **Casos normales:** Al menos 2 por función
- **Casos límite:** Al menos 1 por función
- **Casos de error:** Para funciones que lanzan excepciones
- **Cobertura:** Al menos 90 % de cobertura de código

5.4. Uso de Assert

Listing 11: Diferentes formas de assert

```
1 # Assert simple  
2 assert resultado == 5  
3  
4 # Assert con mensaje personalizado  
5 assert resultado == 5, "La suma deberia dar 5"  
6  
7 # Assert de comparacion  
8 assert resultado > 0  
9 assert resultado >= 10  
10 assert resultado < 100  
11  
12 # Assert de tipo
```

```
13 assert isinstance(resultado, int)
14 assert isinstance(resultado, float)
15
16 # Assert de pertenencia
17 assert resultado in [1, 2, 3, 4, 5]
18 assert "error" in str(excepcion)
19
20 # Assert de valores aproximados (para floats)
21 assert abs(resultado - 3.14159) < 0.00001
22 # O mejor con pytest:
23 assert resultado == pytest.approx(3.14159)
```

5.5. Uso de pytest.raises

Listing 12: Verificar excepciones

```
1 import pytest
2
3 # Verificar que se lanza excepcion
4 def test_lanza_error():
5     with pytest.raises(ValueError):
6         funcion_que_falla()
7
8 # Verificar mensaje de error
9 def test_mensaje_error():
10     with pytest.raises(ValueError, match="debe ser positivo"):
11         factorial(-5)
12
13 # Capturar excepcion para mas verificaciones
14 def test_error_completo():
15     with pytest.raises(ValueError) as exc_info:
16         dividir(10, 0)
17
18     assert "cero" in str(exc_info.value)
```

5.6. Tests Parametrizados (Avanzado - Opcional)

Listing 13: Tests parametrizados con pytest

```
1 import pytest
2
3 @pytest.mark.parametrize("a, b, esperado", [
4     (2, 3, 5),
5     (0, 0, 0),
6     (-1, 1, 0),
7     (100, 200, 300),
8     (-5, -3, -8),
9 ])
10 def test_sumar_parametrizado(a, b, esperado):
11     """Un test que se ejecuta 5 veces con diferentes valores"""
12     assert sumar(a, b) == esperado
13
```

```
14 # Test de es_primo parametrizado
15 @pytest.mark.parametrize("n", [2, 3, 5, 7, 11, 13, 17, 19, 23])
16 def test_numeros_primos(n):
17     """Verifica que numeros primos son identificados"""
18     assert es_primo(n) == True
19
20 @pytest.mark.parametrize("n", [4, 6, 8, 9, 10, 12, 15, 16, 18])
21 def test_numeros_no_primos(n):
22     """Verifica que numeros no primos son identificados"""
23     assert es_primo(n) == False
```

6. Ejemplo de Test-Driven Development

6.1. Paso 1: Escribir el Test Primero

Listing 14: Test primero (falla porque funcion no existe)

```
1 # test_calculadora.py
2 from calculadora import mcd # Funcion que aun no existe
3
4 def test_mcd_12_y_8():
5     """Maximo comun divisor de 12 y 8 debe ser 4"""
6     assert mcd(12, 8) == 4
7
8 # Al ejecutar: pytest -> ERROR: cannot import 'mcd'
```

6.2. Paso 2: Implementar Función Mínima

Listing 15: Implementacion minima

```
1 # calculadora.py
2 def mcd(a, b):
3     """Calcula el maximo comun divisor"""
4     while b:
5         a, b = b, a % b
6     return a
7
8 # Al ejecutar: pytest -> PASSED
```

6.3. Paso 3: Agregar Más Tests

Listing 16: Mas tests para cubrir casos

```
1 def test_mcd_casos_varios():
2     assert mcd(12, 8) == 4
3     assert mcd(20, 15) == 5
4     assert mcd(17, 13) == 1
5     assert mcd(100, 50) == 50
6
7 def test_mcd_con_cero():
8     assert mcd(5, 0) == 5
```

```
9 | assert mcd(0, 5) == 5
```

7. Consejos y Recomendaciones

- **Un test, un concepto:** Cada test debe verificar UNA cosa específica
- **Nombres descriptivos:** `test_factorial_de_numero_negativo_lanza_error`
- **Documenta los tests:** Usa docstrings para explicar qué verifica cada test
- **Ejecuta tests frecuentemente:** Después de cada cambio
- **Tests independientes:** Cada test debe poder ejecutarse solo
- **Casos de error importan:** Verifica que los errores se manejan correctamente
- **Cobertura no es todo:** 100 % cobertura no significa código perfecto
- **Red-Green-Refactor:** Test falla (red) → Código pasa (green) → Mejorar (refactor)
- **Automatiza:** Los tests deben ejecutarse automáticamente

8. Entregables

1. `calculadora.py` - Todas las funciones implementadas
2. `test_calculadora.py` - Al menos 40 tests
3. `requirements.txt` - Dependencias del proyecto
4. `README.txt` - Instrucciones de instalación y ejecución
5. `reporte_cobertura.txt` - Captura del reporte de cobertura
6. TODOS los tests deben pasar (0 fallos)
7. Cobertura mínima: 90 %

9. Defensa del Código (60 % de la nota final)

9.1. ¿Qué es la defensa del código?

La defensa del código es una reunión virtual donde presentarás y explicarás tu trabajo. Esta defensa es **obligatoria** y representa el **60 % de la nota final** de esta tarea.

9.2. ¿Cómo funciona?

Durante la defensa deberás:

1. **Compartir tu pantalla** mostrando código y terminal.
2. **Ejecutar los tests en vivo:**
 - `pytest -v` para ver todos los tests
 - `pytest --cov=calculadora` para ver cobertura
 - Demostrar que TODOS los tests pasan
 - Abrir reporte HTML de cobertura
3. **Explicar tus tests:**
 - Por qué creaste cada tipo de test
 - Qué verifica cada assert
 - Cómo elegiste los casos de prueba
 - Por qué ciertos valores son casos límite
4. **Demostrar TDD (si aplicaste):**
 - Escribir un nuevo test que falle
 - Implementar la función para que pase
 - Mostrar el proceso red-green-refactor
5. **Responder preguntas como:**
 - “¿Por qué es importante hacer testing?”
 - “¿Qué diferencia hay entre un caso normal y un caso límite?”
 - “¿Cómo funciona `pytest.raises?`”
 - “¿Qué significa 90 % de cobertura?”
 - “¿Rompe un test intencionalmente, qué pasa?”
 - “¿Cómo verificas excepciones con mensajes específicos?”
 - “¿Qué es TDD y cuáles son sus ventajas?”
6. **Demostrar comprensión de:**
 - Propósito de las pruebas unitarias
 - Estructura de un test (Arrange-Act-Assert)
 - Tipos de casos de prueba
 - Interpretación de reportes de pytest
 - Concepto de cobertura de código

9.3. Preguntas Típicas de la Defensa

Prepárate para responder:

- ¿Por qué hacer testing? ¿No basta con probar manualmente?
- ¿Qué es un test unitario?
- ¿Qué hace `assert`?
- ¿Cómo se usa `pytest.raises`?
- ¿Qué es un caso límite? Dame ejemplos
- ¿Qué significa que un test pase?
- ¿Qué es cobertura de código?
- ¿100 % cobertura significa código sin bugs?
- ¿Qué es TDD?
- ¿Cómo instalaste pytest?

9.4. Consejos para una Buena Defensa

- **Ejecuta antes:** Verifica que todos los tests pasan antes de la defensa
- **Conoce tus tests:** Entiende cada test que escribiste
- **Practica romper tests:** Cambia código y observa qué tests fallan
- **Entiende pytest:** Lee la documentación básica
- **Cobertura visual:** Abre el HTML de cobertura y navégalo

9.5. Distribución de la Nota

- **Código funcional (40 %):** Funciones y tests implementados correctamente
- **Defensa del código (60 %):** Tu capacidad de explicar testing

Importante: Si no asistes a la defensa o no puedes explicar tus tests, la nota máxima será 40/100.

10. Desafíos Opcionales (Puntos Extra)

1. Tests parametrizados (+5 pts):

- Usar `@pytest.mark.parametrize` en al menos 5 tests
- Demostrar comprensión de parametrización

2. Fixtures de pytest (+6 pts):

- Crear fixtures con `@pytest.fixture`
- Usar fixtures para datos de prueba reutilizables

- Ejemplo: fixture con lista de números para tests de promedio/mediana

3. Más funciones matemáticas (+7 pts):

- Implementar: MCD, MCM, fibonacci, es_perfecto
- Crear tests completos para cada una (10+ tests por función)

4. Cobertura 100 % (+4 pts):

- Lograr exactamente 100 % de cobertura de código
- Incluir captura de pantalla del reporte

5. Pytest con markers (+5 pts):

- Usar `@pytest.mark.slow` para tests lentos
- Usar `@pytest.mark.skip` con razón
- Ejecutar solo ciertos markers

6. Tests de integración (+6 pts):

- Crear función que usa múltiples operaciones
- Tests que verifican interacción entre funciones
- Ejemplo: calculadora de ecuación cuadrática

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y puedes explicarla en la defensa.

11. Rúbrica de Evaluación

11.1. Distribución General

- **Código Funcional:** 40 % (40 puntos)
- **Defensa del Código:** 60 % (60 puntos)
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +33 puntos adicionales

11.2. Parte 1: Código Funcional (40 puntos)

11.2.1. 1.1 Implementación de Funciones (15 puntos)

- **Operaciones básicas (4 pts):**
 - 4 pts: Suma, resta, multiplicación, división correctas
 - 3 pts: 3 de 4 correctas
 - 2 pts: 2 de 4 correctas
 - 0 pts: Menos de 2 correctas
- **Operaciones avanzadas (6 pts):**
 - 6 pts: Potencia, raíz, factorial, es_primo correctas
 - 5 pts: 3 de 4 correctas
 - 3 pts: 2 de 4 correctas
 - 0 pts: Menos de 2 correctas
- **Operaciones con listas (5 pts):**
 - 5 pts: Promedio, mediana, máximo, mínimo correctas
 - 4 pts: 3 de 4 correctas
 - 2 pts: 2 de 4 correctas
 - 0 pts: Menos de 2 correctas

11.2.2. 1.2 Calidad de Tests (25 puntos)

- **Cantidad de tests (5 pts):**
 - 5 pts: 40+ tests diferentes
 - 4 pts: 30-39 tests
 - 3 pts: 20-29 tests
 - 1 pt: 10-19 tests
 - 0 pts: Menos de 10 tests
- **Cobertura de casos (8 pts):**
 - 8 pts: Tests cubren casos normales, límite y errores para todas las funciones
 - 6 pts: Buena cobertura pero falta en algunas funciones

- 4 pts: Cobertura básica
- 0 pts: Cobertura insuficiente
- **Uso correcto de assert (4 pts):**
 - 4 pts: Todos los assert son apropiados y verifican correctamente
 - 3 pts: Mayoría correctos
 - 1 pt: Uso básico
 - 0 pts: No usa assert correctamente
- **Tests de excepciones (4 pts):**
 - 4 pts: Usa pytest.raises correctamente para verificar errores
 - 3 pts: Usa en algunos casos
 - 1 pt: Uso incorrecto
 - 0 pts: No verifica excepciones
- **Cobertura de código (4 pts):**
 - 4 pts: 90 %+ de cobertura
 - 3 pts: 80-89 % de cobertura
 - 2 pts: 70-79 % de cobertura
 - 1 pt: 60-69 % de cobertura
 - 0 pts: Menos de 60 %

11.3. Parte 2: Defensa del Código (60 puntos)

11.3.1. 2.1 Comprensión de Testing (30 puntos)

- **Concepto de testing (8 pts):**
 - 8 pts: Explica perfectamente por qué hacer testing y sus beneficios
 - 6 pts: Explicación correcta con pequeñas dudas
 - 3 pts: Comprensión básica
 - 0 pts: No entiende el propósito del testing
- **Tipos de casos de prueba (8 pts):**
 - 8 pts: Explica claramente casos normales, límite y excepciones con ejemplos
 - 6 pts: Explica correctamente con alguna confusión
 - 3 pts: Explicación superficial
 - 0 pts: No diferencia tipos de casos
- **Uso de pytest (8 pts):**
 - 8 pts: Explica assert, pytest.raises, ejecución de tests
 - 6 pts: Explica correctamente lo básico
 - 3 pts: Conocimiento limitado

- 0 pts: No entiende pytest

■ **Cobertura de código (6 pts):**

- 6 pts: Explica qué es, cómo medirla, qué significa
- 4 pts: Comprensión básica
- 2 pts: Explicación vaga
- 0 pts: No entiende cobertura

11.3.2. 2.2 Demostración Práctica (20 puntos)

■ **Ejecución de tests (8 pts):**

- 8 pts: Ejecuta tests con diferentes opciones, todos pasan
- 6 pts: Ejecuta correctamente pero algún test falla
- 3 pts: Dificultades para ejecutar
- 0 pts: No puede ejecutar tests

■ **Reporte de cobertura (4 pts):**

- 4 pts: Genera y explica reporte de cobertura HTML
- 3 pts: Genera reporte con ayuda
- 1 pt: Dificultades
- 0 pts: No puede generar reporte

■ **Respuesta a preguntas técnicas (8 pts):**

- 8 pts: Responde correctamente todas las preguntas sobre testing
- 6 pts: Responde la mayoría correctamente
- 3 pts: Respuestas parciales
- 0 pts: No puede responder preguntas básicas

11.3.3. 2.3 Calidad y Organización (10 puntos)

■ **Nombres de tests descriptivos (3 pts):**

- 3 pts: Todos los tests tienen nombres claros y descriptivos
- 2 pts: Mayoría con buenos nombres
- 1 pt: Nombres poco claros
- 0 pts: Nombres confusos

■ **Documentación de tests (4 pts):**

- 4 pts: Todos los tests con docstrings explicativos
- 3 pts: Mayoría documentados
- 1 pt: Documentación mínima
- 0 pts: Sin documentación

■ **Organización del código (3 pts):**

- 3 pts: Tests bien organizados, agrupados lógicamente
- 2 pts: Organización aceptable
- 1 pt: Poco organizado
- 0 pts: Desorganizado

11.4. Puntos Extra (Hasta +33 puntos)

- **Tests parametrizados (+5 pts):** Uso correcto de `@pytest.mark.parametrize`
- **Fixtures (+6 pts):** Creación y uso de fixtures para datos de prueba
- **Más funciones matemáticas (+7 pts):** 4+ funciones adicionales con tests
- **Cobertura 100 % (+4 pts):** Cobertura completa del código
- **Pytest markers (+5 pts):** Uso de markers (slow, skip, etc.)
- **Tests de integración (+6 pts):** Tests que verifican múltiples funciones

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y el estudiante puede explicarla completamente durante la defensa.

11.5. Penalizaciones

- **-100 % de la defensa:** No asiste sin justificación (nota máxima = 40/100)
- **-50 % de la defensa:** Evidencia de no entender los tests que escribió
- **-10 pts:** Algún test falla al ejecutar
- **-5 pts:** Entrega tardía de 1 día
- **-10 pts:** Entrega tardía de 2-3 días
- **-20 pts:** Entrega tardía de 4+ días
- **-5 pts:** No incluye requirements.txt
- **-5 pts:** Menos de 40 tests
- **-3 pts:** No incluye reporte de cobertura

11.6. Nota Mínima Aprobatoria

Para aprobar esta tarea necesitas obtener al menos **60 puntos de 100**.

11.7. Escala de Calificación

- **90-100+ pts:** Excelente - Dominio completo de testing
- **80-89 pts:** Muy Bueno - Comprensión sólida de pruebas unitarias
- **70-79 pts:** Bueno - Comprensión aceptable, necesita práctica
- **60-69 pts:** Suficiente - Comprensión básica, necesita refuerzo
- **0-59 pts:** Insuficiente - No aprobado

12. Recursos de Apoyo

- Documentación pytest: <https://docs.pytest.org/>
- Tutorial pytest: <https://docs.pytest.org/en/stable/getting-started.html>
- Pytest-cov: <https://pytest-cov.readthedocs.io/>
- Testing en Python: <https://realpython.com/pytest-python-testing/>
- TDD con Python: <https://realpython.com/python-testing/>

13. Preguntas Frecuentes

P: ¿Por qué hacer testing si puedo probar manualmente?

R: Los tests automáticos son más rápidos, consistentes, y detectan regresiones cuando cambias código.

P: ¿Cómo instalo pytest?

R: `pip install pytest`

P: ¿Cómo ejecuto un solo test?

R: `pytest test_calculadora.py::test_nombre_especifico`

P: ¿Qué significa que un test pasa?

R: Que el assert se cumplió, el resultado fue el esperado.

P: ¿Cómo veo la cobertura?

R: `pytest --cov=calculadora --cov-report=html`

P: ¿100 % cobertura significa sin bugs?

R: No. Significa que todas las líneas se ejecutaron, pero puede haber casos no probados.

P: ¿Qué es un caso límite?

R: Valores en los bordes del rango válido: 0, 1, -1, máximos, mínimos, listas vacías, etc.

P: ¿Cuántos tests son suficientes?

R: No hay número mágico. Debes cubrir casos normales, límite y errores para cada función.

¡Éxito con tus pruebas unitarias!

*Los tests son la red de seguridad del programador.
Aprende a escribirlos bien y programarás con confianza.*